

CLI Usage Examples

Quick Start

```
# Basic usage with default settings (hybrid matcher, 50 repos, deep analysis)
npm start
```

Matcher Selection

Fuzzy Matcher

Uses Topcoder's fuzzy match API to match languages to skills based on string similarity.

```
node dist/cli.js --matcher fuzzy
```

When to use: - Fast execution - Language-based matching - No AI dependencies

Semantic Matcher

Uses Topcoder's semantic search API or AI providers (OpenAI, Ollama) for intelligent matching.

```
# Using Topcoder semantic API (default)
node dist/cli.js --matcher semantic

# Using OpenAI
node dist/cli.js --matcher semantic --ai-provider openai --openai-key sk-...
```

When to use: - Better understanding of context - Match based on project descriptions and topics - AI-powered similarity

Hybrid Matcher (Recommended)

Combines fuzzy and semantic matching with weighted scoring for best results.

```
node dist/cli.js --matcher hybrid
```

When to use: - Most accurate results - Combines multiple signals - Default and recommended option

Performance Tuning

Quick Analysis (Fast)

Skip deep commit/PR analysis for faster results:

```
node dist/cli.js --no-deep-analysis --max-repos 20
```

Deep Analysis (Comprehensive)

Analyze more repos and commits for thorough results:

```
node dist/cli.js --max-repos 100 --max-commits 50
```

Balanced (Default)

Good balance between speed and accuracy:

```
node dist/cli.js --max-repos 50 --max-commits 30
```

Output Options

Display Results

Control how many skills to show:

```
# Show top 20 skills
node dist/cli.js --max-skills 20

# Show top 5 skills
node dist/cli.js --max-skills 5
```

Text Output

Human-readable formatted output (default):

```
node dist/cli.js --output text
```

JSON Output

Machine-readable JSON format:

```
node dist/cli.js --output json
```

Save to File

Save results for later analysis:

```
# Save text output
node dist/cli.js --output text --output-file results.txt

# Save JSON output
node dist/cli.js --output json --output-file analysis.json
```

Verbosity

Normal Mode (Default)

Shows progress and results:

```
node dist/cli.js
```

Verbose Mode

Shows detailed logs including API calls and processing steps:

```
node dist/cli.js --verbose
```

Real-World Scenarios

Scenario 1: Quick Profile Check

Fast analysis for a quick overview:

```
node dist/cli.js \  
  --matcher fuzzy \  
  --max-repos 20 \  
  --no-deep-analysis \  
  --max-skills 10
```

Expected time: 1-2 minutes

Scenario 2: Comprehensive Analysis

Deep dive for accurate skill recommendations:

```
node dist/cli.js \  
  --matcher hybrid \  
  --max-repos 100 \  
  --max-commits 50 \  
  --max-skills 20 \  
  --verbose \  
  --output json \  
  --output-file full-analysis.json
```

Expected time: 5-10 minutes

Scenario 3: AI-Powered Matching

Use OpenAI for semantic understanding:

```
node dist/cli.js \  
  --matcher hybrid \  
  --ai-provider openai \  
  --openai-key $OPENAI_API_KEY \  
  --max-skills 15 \  
  --output json \  
  --output-file ai-results.json
```

Expected time: 3-5 minutes (with API calls)

Scenario 4: Export for Integration

Generate JSON for integration with other systems:

```
node dist/cli.js \  
  --matcher hybrid \  
  --max-repos 50 \  
  --output json \  
  --output-file integration-data.json
```

```
--output-file topcoder-skills.json
```

Then process the JSON:

```
# Pretty print
cat topcoder-skills.json | jq '.recommendations[] | {skill: .skillName, confidence: .confidence}'

# Filter high-confidence skills
cat topcoder-skills.json | jq '.recommendations[] | select(.confidence > 80)'
```

Troubleshooting

Rate Limiting

If you hit Github API rate limits:

```
# Reduce repos analyzed
node dist/cli.js --max-repos 20 --max-commits 10
```

The tool automatically waits when approaching rate limits.

Long Execution Time

For faster results:

```
node dist/cli.js --no-deep-analysis --max-repos 30
```

Missing Skills

For better coverage:

```
# Use hybrid matcher with more results
node dist/cli.js --matcher hybrid --max-skills 30 --verbose
```

Environment Variables

Create a `.env` file:

```
GITHUB_CLIENT_ID=your_github_oauth_app_client_id
GITHUB_CLIENT_SECRET=your_github_oauth_app_client_secret
OPENAI_API_KEY=sk-your_openai_api_key_optional
```

Or pass directly:

```
OPENAI_API_KEY=sk-... node dist/cli.js --ai-provider openai --matcher semantic
```

Help

View all available options:

```
node dist/cli.js --help
```